RANCANG BANGUN SMART COOLING SYSTEM BERBASIS MIKROKONTROLER ARDUINO DENGAN MONITORING SUHU DAN FITUR EMERGENCY STOP

LAPORAN PROJECT 1

Disusun dan diajukan untuk memenuhi project 1
Mata Kuliah : Sistem Mikrokontroler
Dosen Pengampu : Ucky Pradestha Novettralita, S.Pd., M.Kom

Disusun Oleh :

1.	Dedi Kurniawan	H1H024022
2.	Nursyafika	H1H024023
3.	Nesa Dwi Cahyani	H1H024024
4.	David Ananta Nugraha	H1H024025
5.	Javier Anthonio Justiansah	H1H024026
6.	Yogi Ferdiansyah Amta Miluloh	H1H024027
7.	Dendro Iqbal Pratama	H1H024028

JURUSAN TEKNIK KOMPUTER

FAKULTAS TEKNIK

UNIVERSITAS JENDERAL SOEDIRMAN

PURBALINGGA

APRIL 2026

DAFTAR ISI

DAFTAR ISI ii
BAB I 1
PENDAHULUAN 1
1.1. Latar Belakang 1
1.2. Tujuan 2
BAB II 3
PERANCANGAN SISTEM 3
2.1. Diagram Blok Sistem 3
2.2. Flowchart Sistem 4
2.3. Penjelasan Fungsi Setiap Komponen 5
BAB III 6
IMPLEMENTASI PROGRAM 6
3.1. Listing Program 6
3.2. Penjelasan Program 9
BAB IV 11
HASIL DAN ANALISIS 11
4.1. Hasil 11
4.2. Analisis 11
5.1. Peran Masing-Masing Anggota 14
5.2. Tautan Repository 14
5.3. Tautan Youtube Video Demonstrasi 14

BAB I
PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi yang semakin maju menuntut hadirnya sistem yang mampu bekerja secara otomatis, efisien, serta memiliki tingkat keamanan yang baik, terutama dalam pengendalian suhu pada perangkat elektronik. Suhu yang tidak terjaga dengan baik dapat menurunkan performa bahkan menimbulkan kerusakan pada komponen. Dalam kondisi tertentu, peningkatan suhu yang berlebihan atau overheating dapat menyebabkan gangguan operasional hingga kerusakan permanen pada sistem.

Banyak sistem pendingin yang masih menggunakan metode konvensional dan bergantung pada pengaturan manual. Pendekatan ini memiliki beberapa keterbatasan, seperti kurang tanggap terhadap perubahan suhu yang terjadi secara cepat serta tidak mampu melakukan pemantauan secara berkelanjutan. Hal ini menyebabkan potensi overheating sering tidak terdeteksi lebih awal. Selain itu, sistem pendingin yang tidak terkontrol dengan baik juga dapat mengakibatkan pemborosan energi karena perangkat tetap bekerja meskipun suhu sudah berada dalam kondisi normal.

Seiring berkembangnya teknologi mikrokontroler, tersedia solusi yang lebih efisien untuk mengatasi permasalahan tersebut melalui penerapan sistem pendingin cerdas berbasis Arduino. Sistem ini memungkinkan pemantauan suhu dilakukan secara real-time dengan bantuan sensor, sehingga setiap perubahan suhu dapat segera diketahui dan ditindaklanjuti secara otomatis. Dengan demikian, proses pendinginan dapat berjalan lebih optimal sekaligus meningkatkan efisiensi penggunaan energi.

Di samping efisiensi, aspek keamanan juga menjadi hal yang sangat penting dalam perancangan sistem pendingin. Oleh karena itu, diperlukan fitur tambahan berupa emergency stop yang berfungsi untuk menghentikan sistem secara cepat saat terjadi kondisi darurat, seperti lonjakan suhu yang melampaui batas aman. Fitur ini berperan penting dalam mencegah kerusakan yang lebih serius serta menjaga keselamatan peralatan dan pengguna.

Maka perlunya dirancang sebuah Smart Cooling System berbasis mikrokontroler Arduino yang dilengkapi dengan monitoring suhu dan fitur emergency stop. Sistem ini diharapkan mampu menjadi solusi yang lebih unggul dibandingkan sistem konvensional, baik dari sisi efisiensi energi, kecepatan respons, maupun tingkat keamanan dalam penggunaannya.

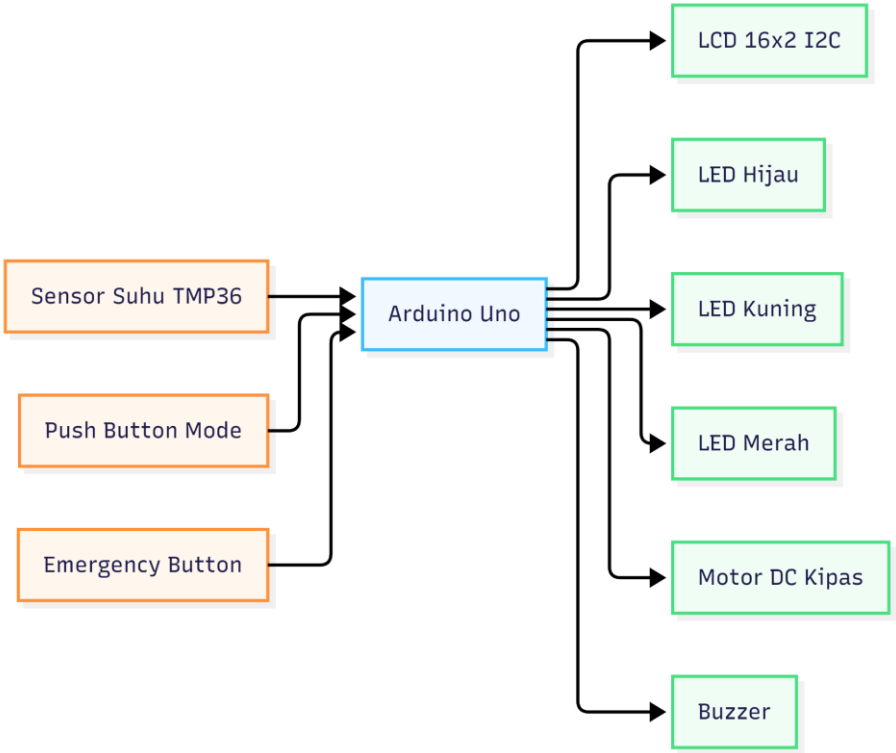
1.2. Tujuan

Adapun tujuan dari pembuatan Smart Cooling System berbasis Arduino ini adalah sebagai berikut:

1. Mengimplementasikan pembacaan suhu menggunakan sensor TMP36 sebagai input sistem.
2. Mengendalikan kecepatan kipas secara otomatis menggunakan metode PWM berdasarkan nilai suhu.
3. Menggunakan LED sebagai indikator kondisi suhu di beberapa kategori.
4. Mengaktifkan buzzer sebagai alarm saat suhu mencapai kondisi lebih dari sama dengan 35 derajat celcius.
5. Mengimplementasikan fitur emergency stop menggunakan interrupt untuk mematikan sistem secara otomatis

BAB II
PERANCANGAN SISTEM

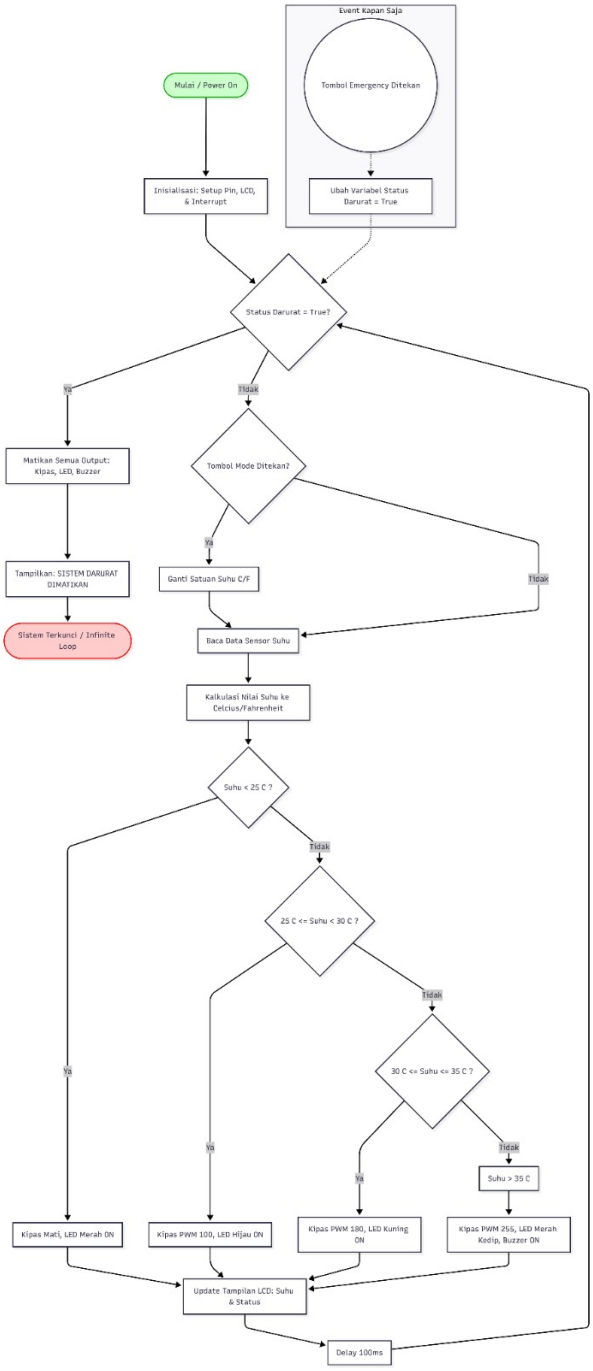
2.1. Diagram Blok Sistem



Gambar 2.1 Diagram Blok Smart Cooling System

Diagram ini menggambarkan arsitektur Smart Cooling System di mana Arduino Uno bertindak sebagai otak pusat yang memproses data dari tiga input utama, yaitu sensor suhu TMP36, tombol mode, dan tombol darurat. Berdasarkan input tersebut, Arduino akan menginstruksikan berbagai output untuk bekerja secara sinkron, menampilkan informasi pada LCD 16x2, memberikan sinyal visual melalui tiga tingkatan LED (hijau, kuning, merah), mengaktifkan motor kipas untuk pendinginan, serta membunyikan buzzer sebagai peringatan suara.

2.2. Flowchart Sistem



Gambar 2.2 Flowchart Smart Cooling System

Smart Cooling Sistem bekerja dengan melakukan inisialisasi awal dan pemantauan fitur darurat yang berfungsi untuk mematikan seluruh output secara otomatis jika diaktifkan. Pada kondisi standar, sistem membaca sensor suhu dan menyediakan fitur penggantian satuan unit (C/F) sebelum mengklasifikasikan kondisi suhu ke dalam beberapa tingkatan tertentu. Setiap tingkatan suhu tersebut

akan memicu respons otomatis berupa pengaturan kecepatan kipas serta aktifnya indikator peringatan baik berupa LED maupun buzzer. Hasil pemantauan ini kemudian ditampilkan pada layar LCD dan diperbarui secara berkala dalam interval waktu yang singkat.

2.3. Penjelasan Fungsi Setiap Komponen

Dalam perancangan sistem ini, digunakan beberapa komponen elektronik yang masing-masing memiliki fungsi berbeda:

Tabel 3.1 Komponen yang Digunakan Beserta Fungsinya

No	Nama Komponen	Fungsi Komponen
1.	Arduino Uno	Sebagai pusat pengendali sistem yang memproses input dan mengatur output
2.	Breadboard	Media untuk merangkai komponen tanpa solder
3.	Sensor Suhu TMP36	Mengukur suhu lingkungan dan mengirimkan data analog ke Arduino
4.	LED	Indikator kondisi suhu (rendah, sedang, tinggi)
5.	Resistor 220 ohm	Membatasi arus agar LED tidak rusak
6.	Resistor 1k ohm	Membatasi arus pada transistor npn
6.	Motor DC	Berfungsi sebagai kipas pendingin
7.	Transistor NPN	Sebagai saklar elektronik untuk mengontrol motor dari Arduino
8.	Buzzer	Memberikan alarm saat suhu tinggi
9.	Push Button	Digunakan sebagai tombol emergency stop
10.	LCD I2C	Menampilkan suhu dan status kipas
11.	Kabel Jumper	Menghubungkan antar komponen

BAB III
IMPLEMENTASI PROGRAM

3.1. Listing Program

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

const int pinSuhu      = A0;
const int pinBtnMode   = 8;
const int pinBtnEmg    = 2;
const int pinLedHijau  = 4;
const int pinLedKuning = 5;
const int pinLedMerah  = 6;
const int pinBuzzer    = 7;
const int pinKipas     = 9;

LiquidCrystal_I2C lcd(0x27, 16, 2);
volatile bool statusDarurat = false;
bool modeFahrenheit = false;
bool statusTombolLama = HIGH;
void setup() {
  pinMode(pinSuhu, INPUT);
  pinMode(pinBtnMode, INPUT_PULLUP);
  pinMode(pinBtnEmg, INPUT_PULLUP);
  pinMode(pinLedHijau, OUTPUT);
  pinMode(pinLedKuning, OUTPUT);
  pinMode(pinLedMerah, OUTPUT);
  pinMode(pinBuzzer, OUTPUT);
  pinMode(pinKipas, OUTPUT);

  Serial.begin(9600);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Suhu : ");
  lcd.setCursor(0, 1);
  lcd.print("Kipas: ");
  attachInterrupt(digitalPinToInterrupt(pinBtnEmg), daruratTerjadi, FALLING);
```

```
}

void loop() {
  if (statusDarurat) {
    jalankanProsedurDarurat();
    return;
  }
  bool statusTombolSekarang = digitalRead(pinBtnMode);
  if (statusTombolSekarang == LOW && statusTombolLama == HIGH) {
    modeFahrenheit = !modeFahrenheit;
    delay(50);
  }
  statusTombolLama = statusTombolSekarang;

  int nilaiADC   = analogRead(pinSuhu);
  float tegangan = nilaiADC * 5.0 / 1024.0;
  float suhuC    = (tegangan - 0.5) * 100;
  float suhuTampil = modeFahrenheit ? (suhuC * 9.0 / 5.0) + 32.0 : suhuC;
  char unit = modeFahrenheit ? 'F' : 'C';
  String statusKipas = "";
  int kecepatanKipas = 0;

  if (suhuC < 25) {
    statusKipas = "Mati    ";
    kecepatanKipas = 0;
    digitalWrite(pinLedMerah, HIGH);
    digitalWrite(pinLedHijau, LOW);
    digitalWrite(pinLedKuning, LOW);
    noTone(pinBuzzer);
  }
  else if (suhuC >= 25 && suhuC < 30) {
    statusKipas = "Lambat  ";
    kecepatanKipas = 100;
    digitalWrite(pinLedMerah, LOW);
    digitalWrite(pinLedHijau, HIGH);
    digitalWrite(pinLedKuning, LOW);
    noTone(pinBuzzer);
  }
  else if (suhuC >= 30 && suhuC <= 35) {
```

```
    statusKipas = "Cepat   ";
    kecepatanKipas = 180;
    digitalWrite(pinLedMerah, LOW);
    digitalWrite(pinLedHijau, LOW);
    digitalWrite(pinLedKuning, HIGH);
    noTone(pinBuzzer);
  }
  else {
    statusKipas = "Maksimal";
    kecepatanKipas = 255;
    digitalWrite(pinLedHijau, LOW);
    digitalWrite(pinLedKuning, LOW);
    digitalWrite(pinLedMerah, HIGH);
    tone(pinBuzzer, 1000);
    delay(100);
    digitalWrite(pinLedMerah, LOW);
    noTone(pinBuzzer);
    delay(100);
  }

  analogWrite(pinKipas, kecepatanKipas);
  lcd.setCursor(7, 0);
  lcd.print(suhuTampil);
  lcd.print(" ");
  lcd.print(unit);
  lcd.print("    ");
  lcd.setCursor(7, 1);
  lcd.print(statusKipas);
  delay(100);
}

void daruratTerjadi() {
  statusDarurat = true;
}
void jalankanProsedurDarurat() {
  analogWrite(pinKipas, 0);
  digitalWrite(pinLedHijau, LOW);
  digitalWrite(pinLedKuning, LOW);
  digitalWrite(pinLedMerah, LOW);
```

```
  noTone(pinBuzzer);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("SISTEM DARURAT ");
  lcd.setCursor(0, 1);
  lcd.print("   DIMATIKAN   ");
  while(true);
}
```

3.2. Penjelasan Program

1. Pada bagian awal program, dilakukan pemanggilan library Wire.h dan LiquidCrystal_I2C.h yang berfungsi untuk mengendalikan LCD berbasis I2C. Setelah itu, dilakukan proses deklarasi pin, di mana masing-masing komponen seperti sensor suhu, tombol, LED indikator, buzzer, dan kipas dihubungkan ke pin tertentu pada Arduino.
2. Pada bagian fungsi setup(), program menginisialisasi seluruh pin sesuai perannya sebagai input maupun output. Tombol diatur menggunakan mode INPUT_PULLUP agar pembacaan sinyal lebih stabil. Selain itu, LCD juga diaktifkan dan menampilkan informasi awal berupa suhu serta status kipas. Pada tahap ini, juga diterapkan interrupt pada tombol emergency, sehingga sistem dapat langsung merespons saat tombol ditekan tanpa harus menunggu siklus loop() selesai.
3. Pada fungsi loop(), program dijalankan secara berulang terus-menerus. Pada tahap awal, sistem akan memeriksa apakah kondisi darurat sedang aktif. Jika kondisi tersebut terdeteksi, maka program segera menjalankan prosedur darurat dan menghentikan proses utama yang sedang berjalan. Jika kondisi darurat tidak aktif, sistem membaca status tombol mode untuk menentukan apakah suhu ditampilkan dalam Celsius atau Fahrenheit. Logika ini menggunakan variabel modeFahrenheit yang berubah setiap kali tombol ditekan.
4. Program kemudian menjalankan logika kontrol kipas dengan PWM berdasarkan kategori suhu:
   Suhu < 25°C: kipas mati, LED merah menyala sebagai tanda standby, buzzer tidak aktif.
   1) 25°C ≤ Suhu < 30°C: kipas berputar lambat (PWM 100), LED hijau menyala, buzzer tidak aktif.
   2) 30°C ≤ Suhu ≤ 35°C: kipas berputar cepat (PWM 180), LED kuning menyala, buzzer tidak aktif.
   3) Suhu > 35°C: kipas berputar maksimal (PWM 255), LED merah berkedip, buzzer berbunyi sebagai alarm.

5. Fungsi daruratTerjadi() dipicu oleh interrupt ketika tombol emergency ditekan. Fungsi ini mengubah nilai statusDarurat menjadi true. Fungsi jalankanProsedurDarurat() kemudian menghentikan seluruh output dan menampilkan pesan darurat di LCD, serta menghentikan loop dengan perintah while(true);.

BAB IV
HASIL DAN ANALISIS

4.1. Hasil

Setelah sistem *Smart Cooling System* berbasis Arduino selesai dirakit dan diprogram, dilakukan pengujian untuk melihat apakah setiap komponen sudah bekerja sesuai rancangan. Berdasarkan hasil pengujiannya, sensor suhu TMP36 dapat membaca suhu lingkungan dan menampilkan nilai suhu secara real-time pada LCD. Sistem juga dapat menampilkan suhu dalam dua satuan, yaitu Celsius dan Fahrenheit, yang dapat diubah menggunakan tombol mode.

Kipas DC dapat merespons perubahan suhu sesuai kategori yang sudah ditentukan. Pada suhu di bawah 25°C, kipas dalam kondisi mati, LED merah menyala, dan buzzer tidak aktif. Pada rentang suhu 25°C ≤ suhu < 30°C, kipas berputar dengan kecepatan rendah, LED hijau menyala, dan buzzer tetap tidak aktif. Pada rentang suhu 30°C ≤ suhu ≤ 35°C, kipas berputar lebih cepat, LED kuning menyala, dan buzzer tidak aktif. Sementara itu, ketika suhu lebih dari 35°C, kipas berputar maksimal, LED merah berkedip, dan buzzer berbunyi sebagai tanda peringatan.

LCD dapat menampilkan informasi suhu pada baris pertama dan status kipas pada baris kedua. Tampilan LCD juga berubah sesuai kondisi suhu dan status kipas yang sedang berjalan.

Fitur emergency stop juga berhasil berjalan dengan baik. Ketika tombol emergency ditekan, sistem langsung menghentikan kipas, mematikan seluruh LED, menonaktifkan buzzer, dan menampilkan pesan "SISTEM DARURAT DIMATIKAN" pada LCD. Setelah kondisi darurat aktif, sistem tidak melanjutkan proses utama.

4.2. Analisis

Berdasarkan hasil pengujian, sistem *Smart Cooling System* berbasis Arduino sudah bekerja sesuai dengan tujuan yang dibuat. Sensor TMP36 dapat membaca suhu dan menampilkannya secara real-time, sehingga pengguna bisa mengetahui kondisi suhu secara langsung melalui LCD.

Penggunaan metode PWM pada kipas cukup efektif karena kipas tidak hanya memiliki kondisi mati dan menyala, tetapi juga dapat berputar dengan beberapa tingkat kecepatan. Dengan cara ini, kipas dapat menyesuaikan kerja sesuai suhu yang terdeteksi. Saat suhu masih rendah, kipas tidak perlu menyala sehingga penggunaan energi menjadi lebih hemat. Saat suhu meningkat, kipas akan berputar lebih cepat sesuai kebutuhan.

LED dan buzzer juga membantu pengguna dalam mengetahui kondisi sistem. LED digunakan sebagai tanda visual untuk membedakan kondisi suhu, sedangkan buzzer digunakan sebagai peringatan ketika suhu berada pada kondisi tinggi. Dengan adanya indikator ini, pengguna tidak hanya melihat informasi dari LCD, tetapi juga mendapat tanda melalui lampu dan suara.

Fitur emergency stop menjadi bagian penting dalam sistem ini karena dapat menghentikan sistem saat terjadi kondisi darurat. Penggunaan interrupt membuat sistem dapat langsung merespons ketika tombol emergency ditekan, tanpa harus menunggu proses utama selesai. Hal ini membuat sistem lebih aman karena output seperti kipas, LED, dan buzzer dapat langsung dimatikan.

Namun, sistem ini masih memiliki beberapa kekurangan. Sensor TMP36 masih dapat dipengaruhi oleh kondisi lingkungan dan tegangan, sehingga pembacaan suhu bisa kurang akurat jika belum dikalibrasi. Selain itu, penggunaan LED merah sebagai tanda standby dan juga tanda bahaya dapat membuat pengguna sedikit bingung. Oleh karena itu, pada pengembangan selanjutnya, indikator dapat dibuat lebih jelas agar setiap kondisi memiliki tanda yang berbeda.

Secara keseluruhan, sistem ini sudah dapat menjalankan fungsi utamanya, yaitu membaca suhu, mengatur kecepatan kipas secara otomatis, menampilkan informasi pada LCD, serta menyediakan fitur emergency stop. Sistem ini juga masih bisa dikembangkan lagi, misalnya dengan sensor yang lebih akurat atau fitur pemantauan jarak jauh.

BAB V
KESIMPULAN

Berdasarkan perancangan, pembuatan, dan pengujian yang telah dilakukan, *Smart Cooling System* berbasis Arduino berhasil dibuat sesuai dengan tujuan project. Sistem dapat membaca suhu menggunakan sensor TMP36 dan menampilkan hasilnya secara real-time pada LCD.

Sistem juga dapat mengatur kecepatan kipas secara otomatis menggunakan PWM sesuai dengan kategori suhu. Selain itu, LED dapat digunakan sebagai indikator kondisi suhu, sedangkan buzzer dapat memberikan peringatan ketika suhu berada pada kondisi tinggi.

Fitur emergency stop juga dapat berjalan dengan baik. Ketika tombol emergency ditekan, sistem langsung menghentikan seluruh output dan menampilkan pesan darurat pada LCD. Fitur ini membuat sistem menjadi lebih aman saat terjadi kondisi darurat.

Secara keseluruhan, *Smart Cooling System* yang dibuat sudah mampu menjalankan fungsi utama, yaitu monitoring suhu, mengatur kecepatan kipas, memberikan indikator visual dan suara, serta menyediakan fitur penghentian darurat. Sistem ini dapat menjadi solusi pendinginan otomatis yang lebih efisien dan aman dibandingkan sistem pendingin manual.

BAB VI
PENUTUP

5.1. Peran Masing-Masing Anggota

Tabel 5.1 Peran Masing-Masing Anggota

Nama	NIM	Kontribusi
Dedi Kurniawan	H1H024022	Membuat repository github
Nursyafika	H1H024023	Membuat Hasil, Analisis, Kesimpulan, dan penambahan di beberapa sub bab.
Nesa Dwi Cahyani	H1H024024	Membuat latar belakang, penjelasan program, dan melengkapi daftar isi.
David Ananta Nugraha	H1H024025	Membuat Rangkaian Tinkercad, membuat struktur laporan.
Javier Anthonio Justiansah	H1H024026	Membuat Flowchart, merapikan rangkaian.
Yogi Ferdiansyah Amta Miluloh	H1H024027	Penjelasan fungsi, revisi listing program, tambahan penjelasan program, hasil & analisis
Dendro Iqbal Pratama	H1H024028	Revisi GiHub, Push video presentasi ke youtube

5.2. Tautan Repositori

https://github.com/DebuLangit/smart_cooling_system

5.3. Tautan Youtube Video Demonstrasi

https://youtu.be/4Qg7MTTjftA